UEES / SISTEMAS OPERATIVOS

Sistema de Impresoras en Red

Diseño concurrente y sincronización en Python

Christian Guerrero · Carlos Lema

Situación y actividades concurrentes

Usuarios

Un hilo por usuario envía solicitudes de impresión y recibe respuestas.

Gestor

Un hilo recibe solicitudes y finalizaciones. Mantiene el orden FIFO y asigna trabajos mientras haya impresoras libres.

Impresoras

Un hilo por impresora simula la impresión y avisa al terminar. Distintas impresoras pueden trabajar a la vez mientras llegan nuevos documentos.

El recurso compartido

Capacidad de impresión

El conjunto de impresoras es limitado. Cada dispositivo puede atender un documento a la vez. Los usuarios compiten por esa capacidad y el gestor decide a qué impresora asignar cada solicitud.

Conflicto: si dos hilos ven libre la misma impresora antes de que alguno la reserve, ambos pueden asignarle un documento.

Clases y responsabilidades

Solicitud	**id_solicitud, id_usuario, documento** crear(): prepara los datos del mensaje.
Usuario	**id_usuario y buzones** enviar_solicitud() y recibir_respuesta().
GestorImpresion	**recurso, pendientes y buzones** despachar() reserva; procesar_fin() libera.
RecursoImpresoras	**_registro y _mutex en el ejemplo** reservar() consulta y asigna; liberar() valida y devuelve capacidad.
Impresora	**id_impresora y buzones** imprimir() atiende; notificar_fin() avisa al gestor.

Tipo básico: un contador de cupos

```
from threading import Lock

class CuposImpresion:
    def __init__(self, total):
        self.total = total
        self._disponibles: int = total
        self._mutex = Lock()

    def reservar(self):
        with self._mutex:
            if self._disponibles == 0:
                return False
            self._disponibles -= 1
            return True

    def liberar(self):
        with self._mutex:
            if self._disponibles < self.total:
                self._disponibles += 1
```

Qué guarda

Un entero con la cantidad de impresoras libres.

Sección crítica

Comprobar el cupo y descontarlo bajo el mismo mutex.

Qué no representa

No identifica qué impresora se reservó ni para quién.

Tipo complejo: un registro por impresora

```
class RecursoImpresoras:
    def __init__(self, ids):
        self._registro = {p: None for p in ids}
        self._mutex = Lock()

    def reservar(self, solicitud_id):
        with self._mutex:
            for p, asignada in self._registro.items():
                if asignada is None:
                    self._registro[p] = solicitud_id
                    return p
            return None
```

Qué guarda

Clave: identificador.
Valor: solicitud asignada o None si está libre.

Sección crítica

Leer el estado, validar que sea None y guardar la solicitud sin soltar el mutex.

El ejemplo supone identificadores de solicitud únicos y distintos de None.

Liberación de una impresora

```
    def liberar(self, impresora_id, solicitud_id):
        with self._mutex:
            if impresora_id not in self._registro:
                return False
            if self._registro[impresora_id] != solicitud_id:
                return False
            self._registro[impresora_id] = None
            return True
```

Un FIN solo libera el dispositivo si su identificador existe y la solicitud coincide con la asignación vigente. Un FIN atrasado o repetido no debe borrar la reserva de otro trabajo.

Dos hilos, una misma instancia

```
from threading import Thread

recurso = RecursoImpresoras(["P1"])

def intentar(recurso, solicitud):
    impresora = recurso.reservar(solicitud)
    print(solicitud, impresora)

hilos = [Thread(target=intentar, args=(recurso, s))
         for s in ("S1", "S2")]
for hilo in hilos:
    hilo.start()
for hilo in hilos:
    hilo.join()
```

Una sola creación

El hilo principal crea recurso una vez y pasa la misma referencia a ambos hilos.

Resultado esperado

Uno recibe P1; el otro recibe None. El orden entre S1 y S2 puede variar.

La prueba conserva la primera reserva para comprobar que no se asigna dos veces.

Sección crítica y elección del mutex

Sin protección

A y B leen P1 = libre. A escribe S1; B escribe S2. Ambos creen tener P1, pero el registro conserva solo S2.

Adquisición y liberación

with self._mutex adquiere el Lock al entrar y lo libera al salir, incluso con return o una excepción. Toda consulta, validación y escritura de la reserva o liberación queda dentro del bloque.

Por qué un mutex

Solo un hilo puede modificar el registro a la vez. Un semáforo con N permisos limita la cantidad de trabajos, pero no evita que dos hilos elijan la misma impresora. La impresión ocurre fuera del bloqueo.

Referencia: documentación oficial de Python, threading · objetos Lock.

Decisión para el proyecto

Elegimos el diccionario

Identifica la impresora libre, conserva la solicitud asignada y permite validar la finalización. El contador solo indica cuántos cupos quedan.

El diseño conserva Paso de Mensajes

Usuarios e impresoras se comunican por buzones. El gestor procesa las solicitudes en orden de recepción y es el único que toca el registro.

Dónde aplica el ejemplo con mutex

Los fragmentos muestran cómo proteger una instancia si varios hilos accedieran directamente. En el diseño publicado, las colas sincronizan los mensajes y el gestor serializa las reservas y liberaciones.

Base: Modelo de Comunicación · secciones 4.1 a 4.6 del proyecto.